

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Practico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Materia:	Programación 1
Alumno:	Jorge Puz Benitez
Docente:	Alberto Cortez
Fecha de entrega:	Junio 2026

Indice

1. Marco Teórico	3
1.1. Listas	3
1.2. Diccionarios	3
1.3. Funciones	3.4
1.4. Estructuras Condicionales	4
1.5. Ordenamientos	4
1.6. Estadísticas Básicas	4.5
1.7. Archivos CSV	5
2. Objetivo del Trabajo	5
3. Diseño y caso practico	6
4. Decisiones Técnicas y Arquitectura	7
4.1. Estructura del Programa	7
4.2. Diagrama de Flujo	7
5. Dificultades y Conclusiones	8
6. Bibliografía	9

1. Marco Teórico

En este apartado voy a explicarles los conceptos fundamentales aplicados en este programa de sistema de gestión de países. Cada concepto está respaldado con ejemplos concretos extraídos del propio proyecto.

1.1 Listas

Una lista en Python es una estructura de datos **ordenada y mutable** que permite almacenar múltiples elementos de cualquier tipo dentro de corchetes `[]`. Los elementos pueden ser accedidos mediante su índice, comenzando desde 0. Las listas permiten agregar, modificar y eliminar elementos en tiempo de ejecución, lo que las hace ideales para colecciones de datos dinámicas

Ejemplo de uso en el proyecto:

```
def cargar_paises():
    "Lee el archivo CSV y devuelve una lista de diccionarios."
    paises = [] # lista vacía
```

1.2 Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares **clave: valor**. Se define con llaves `{ }` y permite acceder a cada valor a través de su clave única. Los diccionarios son mutables y especialmente útiles para representar entidades con múltiples atributos, como registros de una base de datos

Ejemplo de uso en el proyecto:

```
nuevo_pais = {
    "nombre": nombre,
    "poblacion": poblacion,
    "superficie": superficie,
    "continente": continente
}
```

1.3 Funciones

Una función es un bloque de código reutilizable que se define con la palabra reservada **def**, seguida del nombre y paréntesis con los parámetros. Las funciones permiten modularizar el código aplicando el principio de **responsabilidad única**: cada función realiza una sola tarea bien definida. Esto mejora la legibilidad y facilita el mantenimiento (Python Software Foundation, 2024).

Ejemplo de uso en el proyecto:

```
def pedir_entero(mensaje, minimo=0):
    "Pide un número entero al usuario y valida que sea válido."
    while True:
        valor = input(mensaje).strip()
        if valor.isdigit() and int(valor) >= minimo:
            return int(valor)
    print(f" Por favor ingrese un número entero válido (mínimo {minimo}).")
```

1.4 Estructuras Condicionales

Las estructuras condicionales permiten ejecutar bloques de código de forma selectiva. En Python se utilizan las palabras reservadas **if**, **elif** y **else**. Son fundamentales para implementar validaciones, control de flujo y toma de decisiones

Ejemplo de uso en el proyecto:

```
if opcion == "1":
    filtrar_por_continente(países)
elif opcion == "2":
    filtrar_por_poblacion(países)
elif opcion == "3":
    filtrar_por_superficie(países)
else:
    print(" Opción inválida.")
```

El menú principal utiliza una cadena de condicionales if/elif/else para dirigir al usuario a la función correspondiente.

1.5 Ordenamientos

Un algoritmo de ordenamiento reorganiza los elementos de una lista segun un criterio definido. El **algoritmo de burbuja (Bubble Sort)** compara pares de elementos adyacentes y los intercambia si estan en el orden incorrecto, repitiendo el proceso hasta que toda la lista queda ordenada

Ejemplo de uso en el proyecto:

```
for i in range(n - 1):
    for j in range(n - 1 - i):
        if clave == "nombre":
            valor_a = lista[j]["nombre"].lower()
            valor_b = lista[j + 1]["nombre"].lower()
        elif clave == "poblacion":
```

1.6 Estadísticas Básicas

Las estadísticas básicas permiten obtener indicadores descriptivos sobre un conjunto de datos. Los indicadores utilizados son: **máximo**, **mínimo** y **promedio** (suma dividida por la cantidad de elementos).

Ejemplo de uso en el proyecto:

```
for p in países:
    total_pob += p["poblacion"]
    total_sup += p["superficie"]

    if p["poblacion"] > mayor_pob["poblacion"]:
        mayor_pob = p
```

Con un único recorrido de la lista se calculan simultáneamente el máximo, el mínimo y la suma para obtener el promedio. Además, se contabiliza la cantidad de países por continente usando un diccionario como contador.

1.7 Archivos CSV

CSV es un formato de archivo de texto plano donde los datos se organizan en filas y columnas separadas por comas. Python incluye el módulo **csv** en su biblioteca estándar. La clase **DictReader** lee cada fila como un diccionario usando la primera fila como nombres de columna (Python Software Foundation, 2024).

Ejemplo de uso en el proyecto:

```
import csv # Modulo para que python pueda leer archivos CSV

ARCHIVO_CSV = "países.csv" # archivo donde guardamos los datos de los países
try:
    with open(ARCHIVO_CSV, newline="", encoding="utf-8") as archivo:
        lector = csv.DictReader(archivo)
        for fila in lector:
```

El archivo países.csv actúa como base de datos del sistema. Al iniciar el programa se carga su contenido en memoria y al salir con la opción 0 se guardan los datos nuevos

2. Objetivo del Trabajo

El objetivo principal de este Trabajo Practico Integrador es desarrollar una aplicación en Python que permita gestionar información sobre países del mundo, integrando todos los conceptos fundamentales aprendidos durante Programación 1.

Los objetivos específicos son:

- Aplicar listas y diccionarios como estructuras de datos principales.
- Modularizar el código utilizando funciones con responsabilidad única.
- Implementar lectura y escritura de archivos CSV para persistencia de datos.
- Desarrollar un sistema de búsqueda, filtrado y ordenamiento de registros.
- Calcular estadísticas descriptivas sobre el dataset.
- Incorporar validaciones de entrada para garantizar la integridad de los datos.

3. Diseño del Caso Practico

El sistema gestiona un dataset de países donde cada registro contiene: **nombre** (string), **poblacion** (entero), **superficie en km2** (entero) y **continente** (string). Los datos se almacenan en **países.csv** y se cargan en memoria al iniciar el programa.

Estructura del archivo CSV:

```

países.csv
1  nombre,poblacion,superficie,continente
2  Chile,19610014,756102,América
3  Colombia,52085168,1141748,América
4  Perú,34049415,1285216,América

```

Menú de opciones del sistema:

Opción	Descripción
1	Agregar un país (con validación de campos vacíos y duplicados)
2	Actualizar poblacion y superficie de un país existente
3	Buscar país por nombre (coincidencia parcial o exacta)
4	Filtrar por continente, rango de poblacion o rango de superficie
5	Ordenar por nombre, poblacion o superficie (ascendente o descendente)
6	Mostrar estadísticas: máximos, mínimos, promedios y conteo por continente
7	Listar todos los países en formato tabla
0	Salir y guardar todos los cambios en el CSV

4. Decisiones Técnicas y Arquitectura

4.1 Estructura modular del código

El programa se organizó en módulos lógicos dentro de un único archivo Python, separados por comentarios que identifican cada sección:

Modulo	Funciones
Datos	cargar_paises(), guardar_paises(), listar_paises()
Validaciones	pedir_entero()
Búsquedas	buscar_pais()
Filtros	filtrar_por_continente(), filtrar_por_poblacion(), filtrar_por_superficie(), menu_filtros()
Ordenamiento	ordenar_burbuja(), menu_ordenamiento()
Estadísticas	mostrar_estadisticas()
Menú Principal	mostrar_menu(), main()

4.2 Diagrama de flujo del programa

El siguiente esquema representa el flujo general de operaciones del sistema:



4.3 Decisiones de diseño destacadas

Persistencia al salir: Los datos se guardan en el CSV únicamente al elegir la opción 0. Esto evita escrituras innecesarias al disco en cada operación.

Copia en ordenamiento: La función `ordenar_burbuja()` trabaja sobre una copia de la lista (`países[:]`) para no alterar el orden original de los datos en memoria.

Validación centralizada: La función `pedir_entero()` centraliza la validación de entradas numéricas, evitando duplicar el mismo código en cada función que lo requiere.

Búsqueda sin distinción de mayúsculas: Todas las comparaciones de texto usan `.lower()` para que la búsqueda y los filtros funcionen independientemente de mayúsculas o minúsculas.

5. Dificultades y Conclusiones

5.1 Dificultades encontradas

Orden de definición de funciones

Python requiere que las funciones estén definidas antes de ser llamadas. El error `NameError` fue el primero en aparecer y llevo a reorganizar todo el código en el orden correcto: primero las funciones auxiliares, luego `main()`, y finalmente el bloque `if __name__`.

El bloque `if __name__ == '__main__':`

Inicialmente este bloque fue colocado dentro de la función `main()` con indentación incorrecta, lo que causaba que el programa se ejecutara sin mostrar el menú. Al comprender que debe ir fuera de toda función, al final del archivo sin indentación, el problema se resolvió.

Modificación de diccionarios en lista

Al implementar `actualizar_pais()`, surgió la duda de si modificar el diccionario encontrado modificaría también la lista original. Se comprendió que en Python los diccionarios se pasan por referencia, por lo que la modificación actualiza directamente el elemento en `países[]`.

Validación de entradas numéricas

Controlar que el usuario ingrese solo números enteros válidos requirió el uso de `.isdigit()` combinado con un bucle `while True`, lo que derivó en la creación de la función auxiliar `pedir_entero()` para reutilizar esa lógica en todo el programa.

5.2 Conclusiones

La realización de este Trabajo Práctico Integrador permitió aplicar de manera práctica los conocimientos aprendidos en Programación 1. A través del desarrollo de un sistema completo.

Dividir el programa en distintas funciones ayudó a mantener el código más ordenado y fácil de entender. Además, cuando surgían errores, resultaba más sencillo encontrarlos y corregirlos.

El uso de listas y diccionarios permitió organizar y almacenar la información de forma clara, facilitando el manejo de los datos dentro del programa. Por último, trabajar con archivos CSV permitió guardar la información para que no se pierda al cerrar el programa, comprendiendo la importancia de conservar los datos para utilizarlos en futuras ejecuciones.

En conclusión, este trabajo sirvió para integrar distintos temas de la materia y fortalecer las habilidades de programación mediante una experiencia práctica y completa.

6. Bibliografía

1. Python Software Foundation. (2024). The Python Tutorial
2. Python Software Foundation. (2024). csv - CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
3. Real Python. (2024). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>